Received XX Month, XXXX; revised XX Month, XXXX; accepted XX Month, XXXX; Date of publication XX Month, XXXX; date of current version XX Month, XXXX.

Digital Object Identifier 10.1109/XXXX.2022.1234567

SmartGuard: A Smart Wearable for Impact and Fall Detection System Prototype

Marcos Moura[1], Marco Chimienti[1], and Agata Breczko[1]

[1]Department of Microsystems, University of South-Eastern Norway, 3184 Borre, Norway

Corresponding author: Marcos Moura (email: marcos.moura@usn.no).

ABSTRACT This work presents the design and implementation of a lightweight, threshold-based wearable fall detection prototype using an MPU-6050 IMU and a Raspberry Pi Pico board. The proposed system focuses on simplicity, accuracy, and real-time operation without depending on external computational resources. The wearable, placed on the waist, captures the acceleration and angular velocity data of the three angles at a sampling rate of 50 Hz. Techniques of calibration and filtering were applied to reduce noise and improve the estimations. The fall detection algorithm uses a multi-step thresholding approach to detect the fall phases: free-fall, impact, posture change, and post-fall stability, with a fall being assigned only after passing through all these steps. A Python-based monitoring interface was developed to visualize real-time data and detection events via serial communication. A series of experiments validated the system in fall and non-fall scenarios. The results demonstrated a successful distinguish between fall and non-fall events and showed reliability on static scenarios, however there was no experiment in a dynamic scenario. Future works aims to improve the hardware, by developing a dedicated PCB with an optimized cover, the software, by improving the threshold algorithm and introduce an machine learning module, and implementing a wireless transmission feature for remote monitoring.

INDEX TERMS Fall detection, Wearable sensors, Human activity recognition, Smart monitoring system.

I. INTRODUCTION

Falls and fall injuries are common and a significant public health issue, and the incidence increases with age [1] due to aspects such as reduced gait speed, delayed recall, use of a walking aid and some deceases [2]. However, the absolute number of fall related injuries are high and independ of age [3]. Norway contains one of the highest percentage of other fall-related injuries requiring health care in Western Europe [4], with over 170.000 falls per year [5], with 70% of those who fall hurting themselves. Neck and hip fractures, head injuries, long-term disability, and the fear of falling again and lost of independence occur following a fall. Around 9000 hip fractures per year happens in Norway due to falls [6] and total of 93.123 hip fractures were identified between 1999 and 2008 in persons over 50 years [7]. This leads to an approximate cost of to the Norwegian state around 1.5 billion NOK [8] and with projections indicating that by 2050, 22% of the global population will be aged 60 and above [9], estimates shows that Norway could face up to 39.45 billion NOK in expenditures related to unintentional injuries such as falls [10]. Falls are also among the most common causes of death in Norway, specially in rural areas [11].

In the Norwegian emergency response system, the issue is equally critical. Nilsbakken et al. [12] reported that elderly trauma patients are significantly more likely to receive under-triaged dispatches, with more than seven times the odds compared to other age groups. This shows the challenges associated with correctly assessing fall severity among older adults.

Beyond public health, falls also represent a big concern in the industry sector. In the Norwegian construction industry, Winge et al. [13] found that the most frequent accidents involve falls or contact with falling objects. Similarly, Holen et al. [14] demonstrated that falls are among the most common types of accidents in Norwegian fishing vessels and fish farms. This highlights that the impact of falls goes beyond healthcare, affecting emergency services and multiple industrial sectors.

This poses an important issue for both sectors, and technologies for automatic fall detection can help to support these vulnerable groups by helping prevent complications related

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

with fall incidents and improve the efficiency of medical care [8], [15]. A fall detection system (FDS), in general terms, is an architecture designed to autonomously detect falls experienced by the user and notify caregivers as soon as these events occur.

FDSs can be mainly categorized into three types based on sensor architecture: vision-based, environmental, and wearable systems [16]. Vision-based systems apply image-processing techniques to video frames, often improved with artificial neural networks and tracking methods [17]–[19]. Environmental systems rely on sensors placed in the surroundings, such as infrared [20] or acoustic [21], but both vision-based and environmental approaches suffer from limited coverage areas and potential blind spots [22].

Wearable systems embed sensors into devices the user wears, like wristbands or helmets. These systems typically uses inertial measurement units (IMUs), including accelerometers, gyroscopes, and magnetometers, together with wireless communication modules for remote monitoring and IoT integration. However, they face challenges, like the high rate of false positives [23], misclassifications of everyday movements, user discomfort due to continuous usage [24], and limited autonomy caused by high energy consumption [25].

To detect falls, algorithms analyze patterns in acceleration, angular velocity, and orientation, and can be grouped into threshold-based, machine-learning-based, and hybrid approaches [26]. Threshold-based algorithms are simple and lightweight but with more tendency to false alarms. Machine-learning methods improve accuracy by identifying complex patterns, but their computational demands often exceed the capabilities of wearable devices. Hybrid algorithms combine the strengths of both approaches, typically using thresholds to pre-filter signals before machine-learning analysis, improving overall efficiency.

This work describes the prototype design and implementation of a proposed wearable FDS using the data from an IMU. The central idea is to make a reliable system with the minimum hardware setup. This work also proposes a fall detection threshold-based algorithm that fuses the data of the accelerometer and the gyroscope to analyze aspects such as free fall, impact, posture change and post fall parameters. The prototype is lightweight, resistant to light impacts and fully scalable.

II. BACKGROUND AND RELATED WORKS

Several research studies have proposed fall prediction and detection algorithms based on IMUs [27]–[29], but they often differ significantly in sensor configuration and placement [30]. The first assignment showed that preferred position to place the wearable the device is on the waist due to its proximity to the body's center of mass [31]. This work aims to develop a simple but efficient device, by using the IMU MPU6050, that contains an accelerometer and a gyroscope.

Several studies have developed a variety of fall detection algorithms, from simple threshold-based [32] to convolutional networks [33] and even Baesyan statistical analysis [34]. In this work, we utilized a threshold-based algorithm designed to run entirely on the microcontroller. By focusing on specific kinematic features, a balance where the system remains lightweight and autonomous, avoiding the latency and energy costs associated with heavy computational models is achieved.

III. PROTOTYPE DESIGN & IMPLEMENTATION

A. WEARABLE SENSING PART

The sensor chosen is the MPU-6050 Tri-Axial accelerometer with an adjustable full-scale range of ±2g, ±4g, ±8g, and ±16g [35], which can be attached comfortably to the user body without distressing the person's routine life activities. In this work, the accelerometer opperates on a full-scale range of ±2g due to the need of a high sensitivity to small accelerations, since the falls, in general, do not involve high-speed movements or large forces [36].

1) SENSOR POSITION

The waist location was chosen due to its proximity to the body's center of mass [31], getting data with less noise when compared to other locations such as the wrist or head. The ideal wearing position of the sensor is shwon in Figure 1(a). The accelerometer sensor is placed inside the shell where the Y-axis of the sensor is always in parallel with the standing human body axis, the effect of small rotations around Y-axis can be neglected and the IMU can be considered aligned with the body if the rotation angle is smaller than 20° [37]. Since the XZ plane is perpendicular to Y-axis, the pitch and roll angles are captured by analyzing the movements related to X and Z-axis respectively, as shown in Figure 1(b).

The pitch and roll values are calculated as shown on equations 1 and 2, respectively.

$$\text{pitch} = \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right) \tag{1}$$

$$\text{roll} = \arctan\left(\frac{a_y}{a_z}\right) \tag{2}$$

Where a_x, a_y and a_z are the raw acceleration captured by the accelerometer. This data is also used to calculate the total sum vector magnitude for the 3-D acceleration, as shown on equation 3 [28].

$$\text{mag} = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{3}$$

2) SENSOR CALIBRATION

The MPU-6050 is factory calibrated, but variations on manufacturing and external conditions can introduce biases and offsets, and calibration is needed to get the best results.

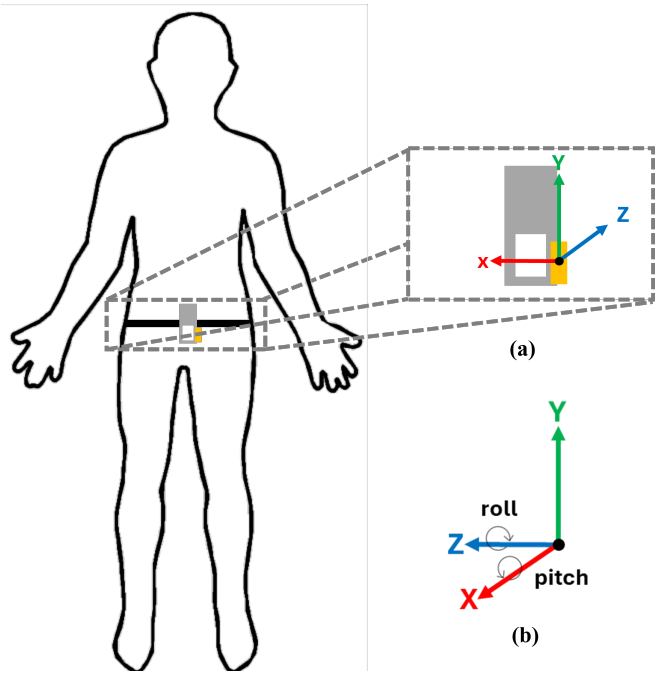


FIGURE 1. **(a) Ideal position of sensing element, where its axes are aligned with the user's body axes and (b) sensor axes and angles captured, Y is the vertical angle, parallel to the standing human body.**

The calibration performed had as objective to correct the accelerometer bias, error in static acceleration measurements, and eliminate the gyroscope drift, that can accumulate over time.

The bias correction of the accelerometer was performed by reading measurements with the sensor in stationary position, with the axes aligned as in Figure 1(a). In the ideal scenario, the sensor should read $0m/s^2$ in X and Z axes, and $9.81m/s^2$ in Y-axis, but the sensor got -0.3m/s^2 in X, -0.2m/s^2 in Z, and $9.01m/s^2$ in Y. The values without offset are calculated as described on equations 4 to 6.

$$a_x = a_{x_raw} + 0.3 \tag{4}$$

$$a_y = a_{y_raw} + 0.9 \tag{5}$$

$$a_z = a_{z_raw} + 0.2 \tag{6}$$

The gyroscope calibration was performed in a similar way, with the sensor in stationary position, without any rotation, the rotation around all the axis should be $0°/s$ in the ideal scenario, however X axis measured -3.5$°/s$ and Z axis measured -1.5$°/s$. The calibrated values are calculated as shown on equations 7 and 8, Y-axis has no offset.

$$\text{gyro}_x = \text{gyro}_{x_raw} + 3.5 \tag{7}$$

$$\text{gyro}_z = \text{gyro}_{z_raw} + 1.5 \tag{8}$$

B. HARDWARE DESIGN

The Raspberry Pi Pico was chosen as processing unit of this prototype due to its reduced size, more memory, and capability of performing more operations per second compared with Arduino Uno [38], [39], the two available out-of-the-shelf for this project.

The full list of components used in the circuit prototype is shown on Table 1. The circuit is composed of two parts: the sensing unit and the alert system. The sensing unit is basically the IMU directly connected to the Raspberry, as shown on the right side of Figure 3. The alert system is composed by a button and a LED, once the fall is detected, the LED will turn on, changing its status only when pressed the button. The button is assembled on a pull-up configuration, to reduce the noise read by the Raspberry's GPIO port.

Component	Function	Quantity
Button	Alert System	1
LED	Alert System	1
470 Ω Resistor	Alert System	2
IMU	Sensing Unit	1
Raspberry Pi Pico	Procesing Unit	1

TABLE 1. **Quantity and functionality of components used in this prototype**

The full schematic is shown on Figure 2. The IMU is connected to the I2C port 0, which is assigned to GPIO 4 for SDA and 5 for SCL. The button is connected to the GPIO 9 and the alert LED on GPIO 16.

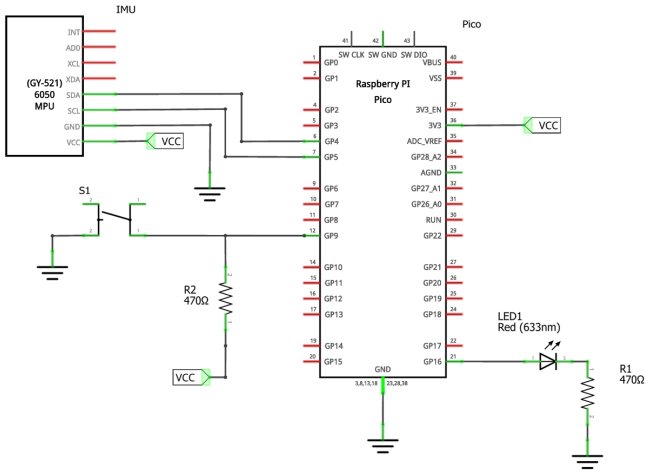


FIGURE 2. **Schematic of prototype**

Once the schematic was defined, the first test round of tests happened in a simple setup on a breadboard, as shown on Figure 3. This assembly ensured the data reliability of the IMU in stationary mode and check if any of the components were malfunctioning. This setup was replaced for a more reliable assembly for the fall simulation tests, since the wire

connections between the IMU and the Raspberry often led to incorrect data readings.

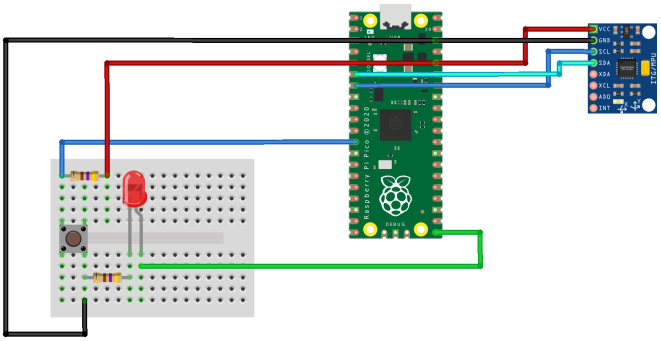


FIGURE 3. Schematic of circuit assembled on breadboard

The final prototype assembly was made by soldering the components of the circuit presented on Figure 2 on a small PCB, with dimensions 2cm x 4.5cm. The final assembly can be seen on Figure 4. The top part of the board contains the alert system, with the connections with Raspberry and the IMU, as well as the resistors soldered below.

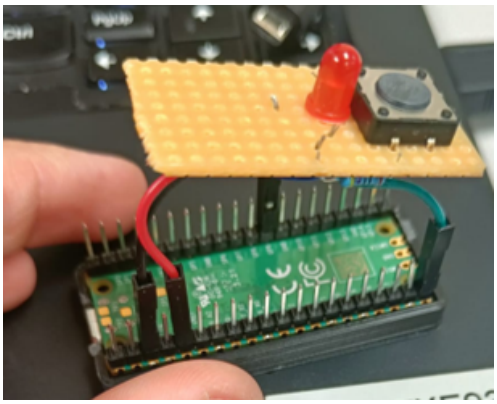


FIGURE 4. Final prototype assembly, with schematic circuit from Figure 2 soldered on a PCB and connected to the Raspberry Pi Pico

Both circuit schematic and breadboard schematic were made using the software Fritzing [40], a very know and user-friendly prototyping tool. The project file is in the Appendix material as *project sketch.fzz*.

C. EMBEDDED SOFTWARE

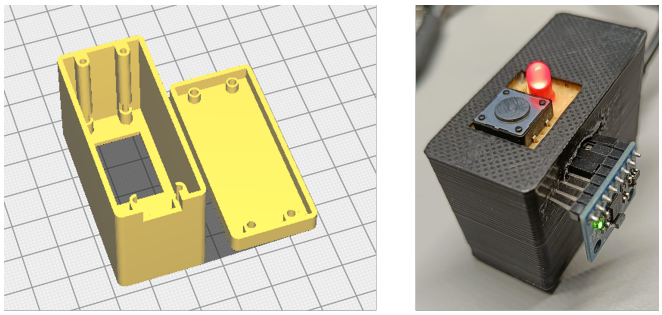
The embedded code of the Raspberry is completely written in C/C++, using the hardware_pio API for GPIO connection and hardware_i2c for the IMU data acquisition [41], [42]. The files that compose the embedded software are available in the Appendix material, as raspberry_code.rar.

The fall-detection algorithm implemented follows a object-oriented approach, encapsulating the components in a C++ class. This class centralizes all data acquisition, filtering, event detection, and the fall detection logic, providing clear separation between hardware access and algorithmic decision layers.

The code also focus on real-time operation and energy efficiency. The timing constraints are handled using the Raspberry's to_ms_since_boot() function, enabling non-blocking delays and precise event-window control.

D. WEARABLE PROTOTYPE

A cover for the prototype was designed to encapsulate all the components, protecting them against impacts and forces related to the experiments and ensuring reliability on connections. The prototype design is shown on Figure 5(a) with dimensions 2.5cm x 5cm x 4 cm. The model was printed using printer Creality Ender-6, the final assembly is shown on Figure 5(b).



(a) (b)

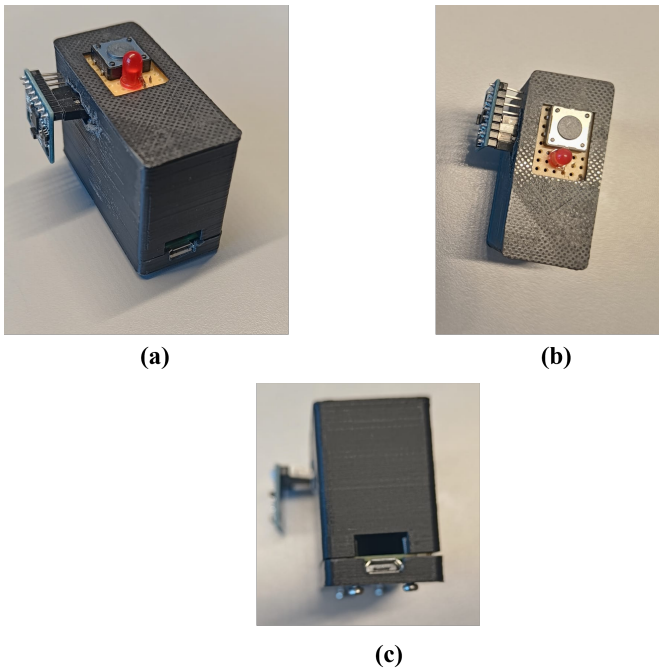
FIGURE 5. Wearable prototype (a) design and (b) assembly

In this first prototype, the IMU is placed outside the cover, to properly align with the cover wall. The opening on the design is for the alert system to be placed, for the LED be easily seen when active and to provide easy access to the button. The three views of the sensor can be seen on Figure 6. The opening for placing the sensor was made after the printing with a plier, and this gives the uneven shape seen on Figure 6(a). The USB opening was printed upside down, and was also required to make an opening at the bottom part, as seen on Figure 6(c). These minor issues did not interfere in the performance of the wearable, but will be corrected for a second version of the printed device.

E. MONITORING INTERFACE

Since the Raspberry Pi Pico board available was not the W version, the transmission module could not be implemented. However, to simulate the information flow that would occur, the raw and processed data is continuously sent in real-time via serial protocol. A simple monitoring interface was built in Python to visualize this data. The interface was built using Tkinter [43], a well known graphical user interface library, and connects with the Raspberry via serial communication, continuously receiving the data. The interface can be seen on Figure 7(a), and shows the data of g force magnitude, top graph, pitch, middle graph, and roll, bottom graph.

The graphs updates as the sensor moves, on the left side, and the flags, on the right, updates on the same flow as



(a) (b) (c)

FIGURE 6. Views from assembled prototype: (a) isometric, (b) alert device, (c) USB connection

the fall detection threshold algorithm, when an impact is detected 7(b) and when a fall is detected 7(c). This interface is an example of a monitoring station that can receive data from the wearable in a wireless transmission. Similar to the embedded software, the monitoring interface was also developed using an oriented-object approach and the files that compose the embedded software are available in the Appendix material as hmi.rar.

IV. FALL DETECTION ALGORITHM

The fall detection algorithm implemented in this is based entirely on a threshold-based method, due to the simplicity to implement and low computational demand [31]. First, the raw data passed through a pre-procesing phase, where two filters were applied to eliminate noite and smooth out the signal. Then, a three steps algorithm detects the transtions from a stactionary position to free-fall, then to impact, and posture change. After that a post-fall monitor triggers a fall if the user does not move again. The IMU data is captured at a rate of 50 Hz.

A. PRE-PROCESSING

Due to the external impacts and sudden changes in acceleration, some noise can appear in the accelerometer's raw data conversion. High-frequency electrical noises also causes variation on readings. Noite also affects the gyroscope and can cause incorrect pitch and roll estimations.

To solve this problem, two filters are applied on the raw data. First, a low-pass filter is applied to smooth out the sensor readings, by eliminating rapid fluctuations, preserving only gradual changes. This filter is applied by making a pondered average of the current reading and the previous one.

The second step is a Kalman filter [44], an adaptative algorithm widely used in MPU-6050 sensor fusion and motion tracking [45], for refining calculation by minimizing uncertainty over time. In this work, the Kalman filter is applied on pitch and roll calculations.

The comparison between the angles before and after both filters can be seen on Figure 8. Both angles, pitch and roll, are smoothed out through the filter. In this prototype, a simple Kalman filter was applied, optimizations such as tuning the filter parameters or using extended Kalman filter were not explored.

B. THRESHOLD-BASED FALL DETECTION

After sampling and pre-processing the acquired datas, the step 1 of the algorithm consists in detecting three possible events: free-fall, impact and posture position by using thresholding methods.

Initially, the system detects free-fall by verifying whether the acceleration magnitude remains below $0.4g$ for at least $200\,\text{ms}$. The acceleration magnitude is defined on equation 3.

The free-fall condition is confirmed if:

$$\text{mag}_g(t) < 0.4 \tag{9}$$

and a free-fall event is confirmed if:

$$t - t_{\text{free}} \geq 200\ \text{ms} \tag{10}$$

Following the free-fall, the algorithm tests whether a high-magnitude peak has occurred by using a fixed threshold of $2.5g$, commonly accepted in the literature as representative of the typical impact phase of a fall, since human activity like walking or running stay below $2g$. This is represented by the equation below:

$$\text{ImpactEvent}(t) = \big(\text{mag}_g(t) > 2.5\big) \tag{11}$$

A rapid change in posture is then analyzed. A lying condition is detected when:

$$|a_z| < 0.5 \quad \wedge \quad |a_x| > 0.5 \tag{12}$$

During the impact-following window of $500\,\text{ms}$, a posture change event occurs if the lying condition changes:

$$\begin{aligned}\text{TiltChangeEvent}(t) = \big(t - t_{\text{tilt}} \leq 500\,\text{ms}\big) \wedge \\ \big(\text{isLyingNow}(t) \neq \text{wasLyingAtImpact}\big)\end{aligned} \tag{13}$$

The algorithm then enters the post-fall monitoring state when at least two of the three previously described events occur simultaneously while the user is lying:

$$\begin{aligned}\text{eventCount} = \\ (\text{FreeFallEvent}) + (\text{ImpactEvent}) + \\ (\text{TiltChangeEvent} \wedge \text{isLyingNow})\end{aligned} \tag{14}$$

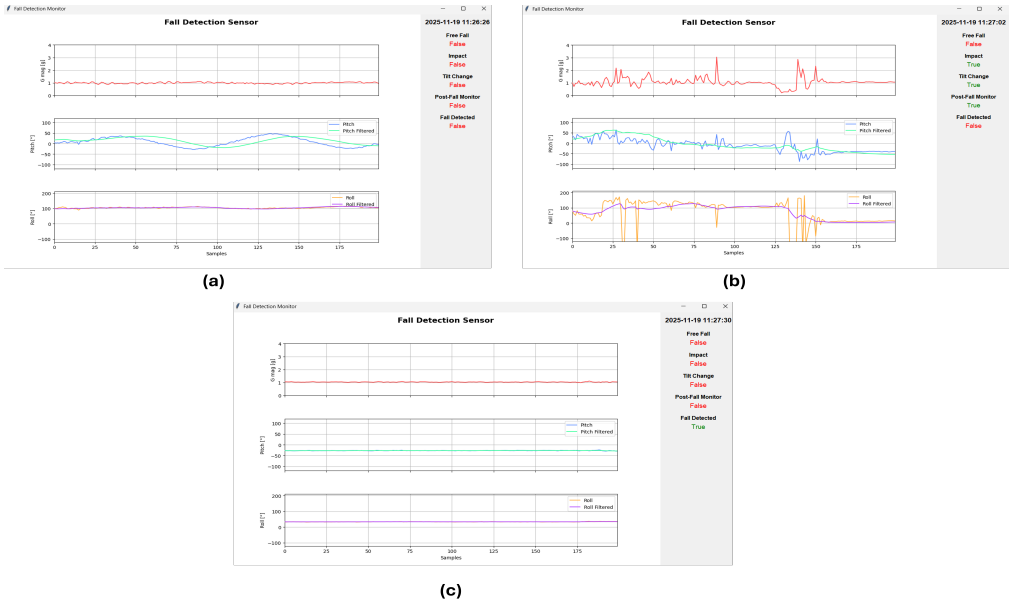


FIGURE 7. **Monitoring user interface in in (a) normal part, (b) beginning of fall and (c) after fall**

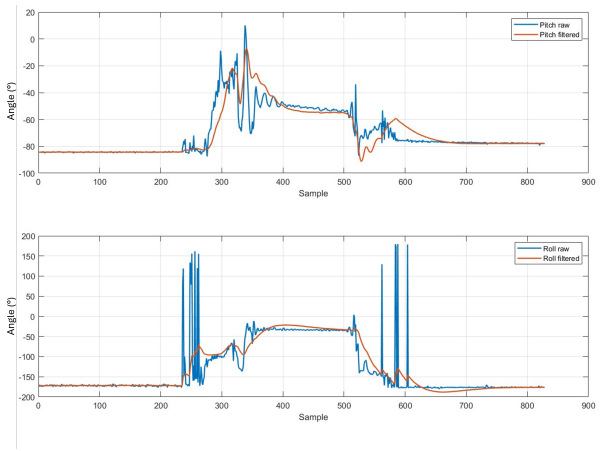


FIGURE 8. **Comparison between the data before and after pre-processing for pitch (top) and roll (bottom) angles.**

When this occurs, the algorithm enters in post-fall monitoring. Finally, the post-fall check consists in verifying, within a 3-second time window, whether the acceleration magnitude stabilizes around $1g$:

$$0.75 \leq \text{mag}_g(t) \leq 1.3 \tag{15}$$

and whether the posture corresponds to a lying state, detected through orientation thresholds, checking if roll value gets lower than 20 or higher than 150, the red areas on Figure 9 and if pitch absolute value goes under 70, the red areas on Figure 10.:

$$\text{isLyingNow}_{\text{post}} = \big(|\text{roll}| < 20^\circ\big) \vee \big(|\text{roll}| > 150^\circ\big) \vee \big(|\text{pitch}| > 70^\circ\big) \tag{16}$$

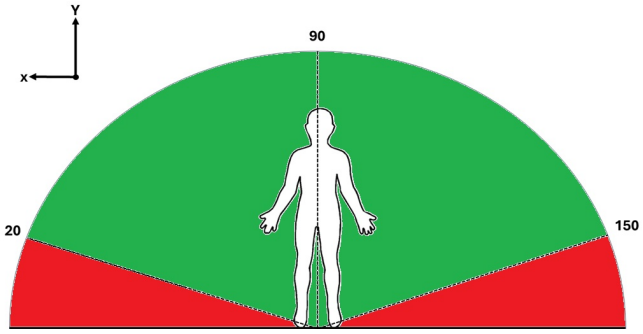


FIGURE 9. **Threshold roll angles**

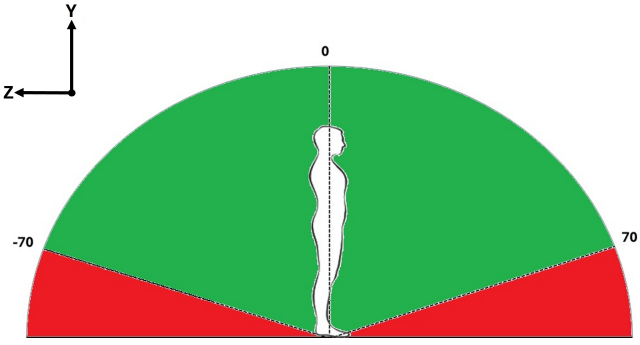


FIGURE 10. **Threshold pitch angles**

A final fall is confirmed if both conditions hold:

$$t - t_{\text{post}} \geq 3000\,\text{ms} \quad \wedge \quad \text{near1g} \quad \wedge \quad \text{isLyingNow}_{\text{post}} \tag{17}$$

When this happens, the fall alert is activated. A manual button also allows the user to mark the fall as a false positive, stopping any alarm transmission.

V. EXPERIMENTS SETUP

To evaluate the reliability and robustness of the proposed FDS, four controlled experiments were conducted using the wearable prototype. The tests covers cover both fall and non-fall scenarios, assuring that the system can correctly identify falls in daily activities. Table 2 summarizes the experimental setup:

TABLE 2. Summary of Experiments Conducted

Experiment	Description	Axis of Rotation	Duration
Lateral Fall	Simulated fall to the side	X-axis	$\approx$ 10s
Frontal Fall	Simulated fall forward	Z-axis	$\approx$ 10s
Sitting	Standing $\rightarrow$ Sitting $\rightarrow$ Standing	Mixed	$\approx$ 15s
Lying Down	Standing $\rightarrow$ Lying on floor	Mixed	$\approx$ 15s

Two simulated falls were tested, a lateral fall over the X-axis, and a frontal fall over the Z-axis. The other tests simulated non fall activities, like sitting and lying, testing if the prototype would trigger a false positive.

The simulated falls took around 10 seconds, enough time to transit from equilibrium to a 1 meter fall and to equilibrium again. The lying simulation took around 15 seconds, due to the smooth rotation a person has when lying. The sitting simulation also took around 15 seconds, and was divided in three steps, first the sensor starts in standing position, than it transits to sit, and after some seconds it stands again.

VI. RESULTS

Figure 11 shows the results of the fall simulation from the XY plane perspective, where the main rotation happens around Z-axis. The fall was simulated in 2 steps, as shown on Figure 11(a), with the standing step going until 145 samples and the falling, followed by the fall, states from step 146 onwards. Figure 11(b) show the angles variation, before and after the Kalman filter, according to the step. The Figure 11(c) shows the acceleration magnitude, with the safe zone marking, values below 0.4g representing free fall and above 2.5 representing an impact. The falling stage was first detected by the sudden drop on acceleration magnitude, followed by a rapid increase, signalizing the detection of an impact.

The fall simulation from the ZY plane perspective, with main rotation around X-axis, are shown on Figure 12. The experiment happens in two steps, as shown on Figure 12, similarly to the previous simulation. The standing step holds for 100 samples, when the fall starts, and the subsequent drop. The angles variation, Figure 12(b), shows a similar result for roll, however the changes on pitch are also noticeable and play a bigger role on assigning a fall. Figure 12(c) shows a similar behavior as in the previous experiment, with the sudden drop on acceleration magnitude before a rapid increase, showing that this behavior is common on falls.

The sitting test was performed in three steps as described on Figure 13(a), first the module is on standing position, after around 150 samples it starts to sit on a office chair, and stays in this position until around 450 samples, then the module moves back to standing position. Figure 13(b) shows the angles variation in this process, as the subject starts sitting, it leans forward and to one of the sides, increasing the roll and decreasing the pitch, then leans backward to place the back on the chair, decreasing the roll and increasing the pitch. During the step two, readings are stationary and mostly constant. When moving up again, the user starts leans forward, increasing roll and decreasing pitch, and then settles in standing position, decreasing roll and increasing pitch. The roll values remained roughly the same on step one, before sitting, and on step three, after standing again. The small disparity on the pitch values when looking at the same points is due to the use of a office swivel chair, and the initial pitch and the final pitch differed.

Figure 13(c) shows the acceleration magnitude across the three steps. Since the sitting movements are usually smoother than a fall, no sudden variations on acceleration magnitude were observed, never passing the threshold.

Figure 14(a) shows the representation of the lying down scenario, with two steps, standing until around 250 samples, when the subject starts laying down. Figure 13(b) shows the angles variation across the experiment. It starts with a similar behavior of the sitting test, with an increase on roll and decrease of pitch as the subject moves onward to first sit more comfortably. Then the roll drops and the pitch increases, above the initial value.

The acceleration magnitude, as shown on Figure 13(c), similarly to the sitting test, does not show any sudden variation, always remaining with values close to 1g.

VII. DISCUSSION

The results from the fall detection tests demonstrate that the system can effectively distinguish between falls and non falls activities. Figure 11, which simulates a lateral fall along the XY plane, clearly shows a change only in roll, and not in pitch, since it wasn't affected in the fall, demonstrating the correct expected behavior. Acceleration magnitude goes below the 0.4g threshold during the free fall, and then above 2.5g during the impact, stabilizing afterwards and confirming the occurrence of a fall.

Figure 12 tested a fall along emphasizing pitch changes. Change in roll were also observed due to the fall happening in both YX and YZ plane, but showing higher influence in pitch, as expected. The acceleration plot follows the same trend as the previous fall test.

In contrast, the tests simulating non fall activities, such as sitting (Figure 13) and lying down (Figure 14), demonstrated

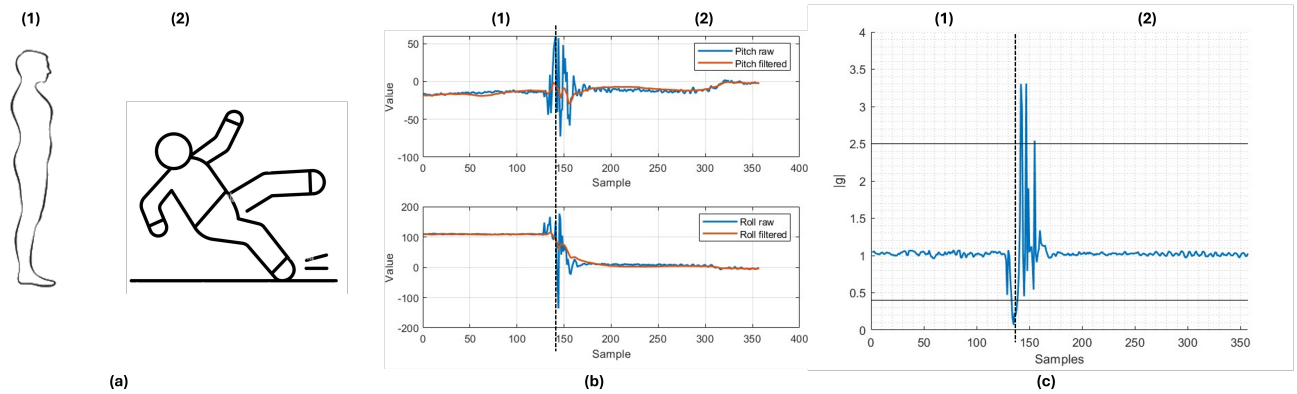


FIGURE 11. **Fall simulation: (a) setup with two steps: (1) stand, (2) fall over Z-axis; (b) raw and filtered data of pitch and roll on the two steps; (c) magnitude of g on the two steps.**

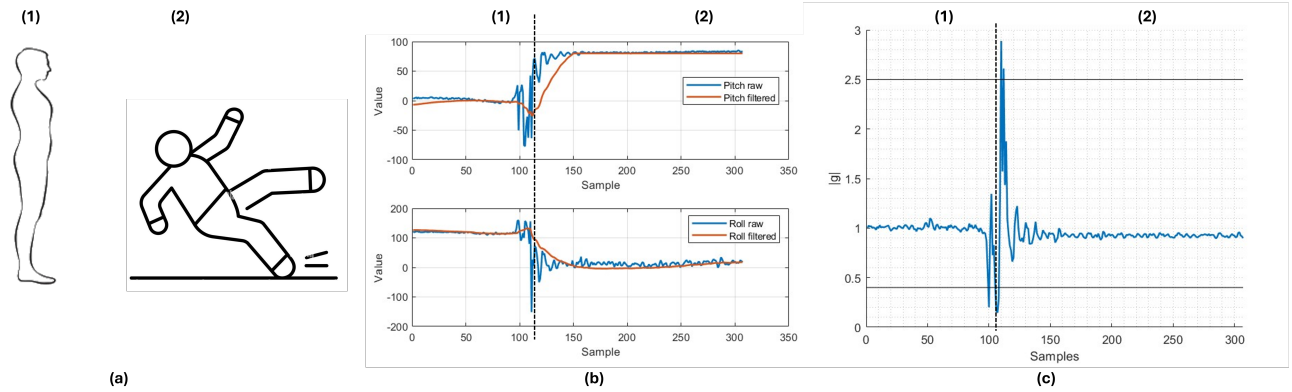


FIGURE 12. **Fall simulation: (a) setup with two steps: (1) stand, (2) fall over X-axis; (b) raw and filtered data of pitch and roll on the two steps; (c) magnitude of g on the two steps.**

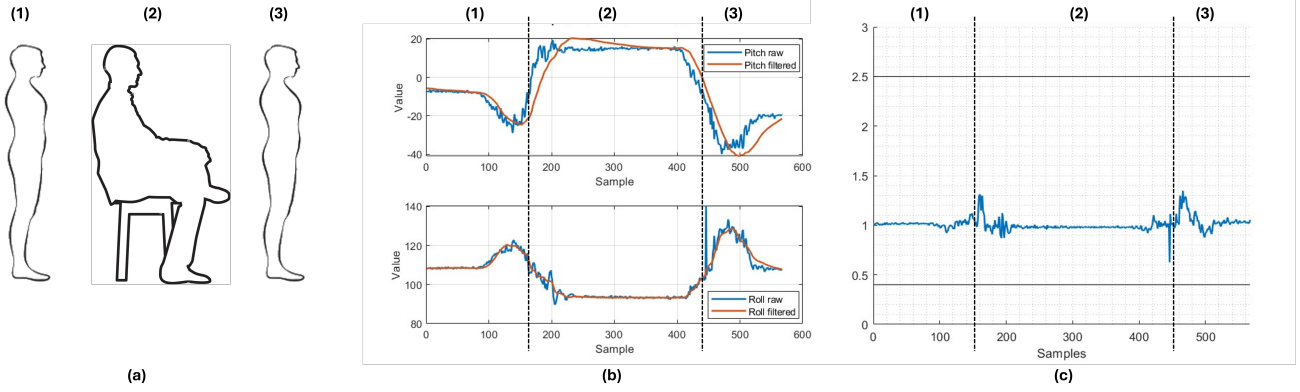


FIGURE 13. **Sitting simulation: (a) setup with three steps: (1) stand, (2) sitting, (3) standing again; (b) raw and filtered data of pitch and roll on the three steps; (c) magnitude of g on the three steps.**

that the system can avoid false positives. For sitting, the acceleration doesn't exceed the threshold values. While pitch and roll result in very small change due to a small movement of the sensor, since ideally, they should not have changed. Similarly, for the lying down test, as expected pitch resulted in a more relevant change than roll, since the lying primarly interested the YZ plane. The magnitude did not exceed the threshold.

Overall, the prototype satisfied all the tests, indicating that the fall detection system performs well in real world

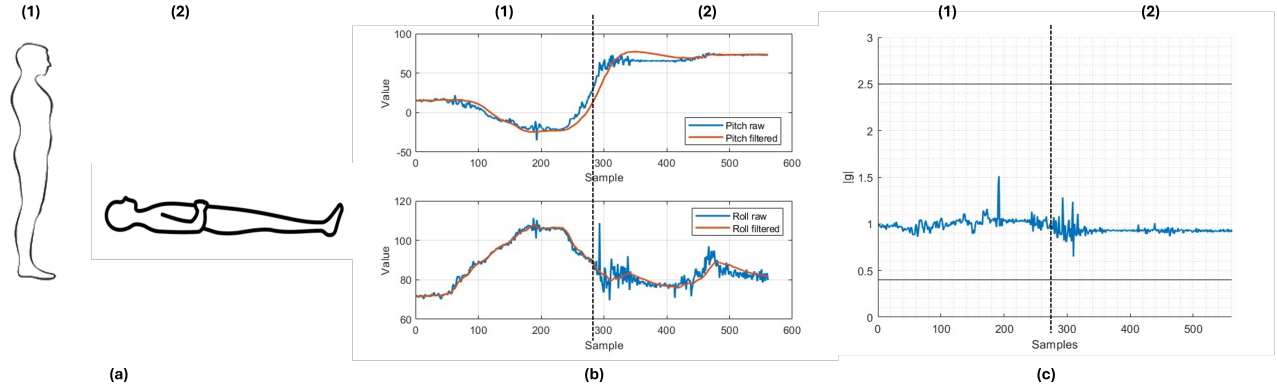


FIGURE 14. **Lying simulation: (a) setup with two steps: (1) stand, (2) lying down; (b) raw and filtered data of pitch and roll on the two steps; (c) magnitude of g on the two steps.**

scenarios, with the biggest limiting factors for the prototype being the sensor placed outside the module, making it susceptive to small undesired movement, resulting in the small deviations to the expected results.

A. LIMITATIONS

The fall detection system prototype, although effective in many scenarios, showed some limitation regarding testing scenarios, design, and detection algorithm. The sensor was positioned externally to the module, introducing a slight instability and causing little data deviations. Possibly, if the sensor had been securely positioned inside the module, the results could have been more accurate. The testing scenarios were only four and, even if enough for describing typical human activity, did not analyze the prototype's behavior during other daily activities such as walking, running, jumping or climbing the stairs. Additionally, even if the fall detection algorithm was functional, it did not always detect free fall. This inconsistency may be due to the short testing distance of around $20cm$ drop. In real life scenarios, falls involve a longer distance and are more dynamic, which would provide the algorithm more time for detecting a free fall. Another limitation is the 3 seconds time for the post fall check, that has shown to be too much resulting in a slower detection time.

B. FUTURE WORKS

The prototype showed promising results, however, due to the simplicity of the detection algorithm and the hardware, several improvements and extensions are planned for future iterations. First, the current prototype uses a basic PCB and an external sensor placement. The second version of this project will include a properly designed and printed PCB and a cover with the manual changes implemented on this one, to ensure a better protection against external impacts. This will also improve the alignment of the sensor with the user's body axes.

The next versions will also feature a wireless communication module, to enable real-time data transmission to a live server. This will allow remote monitoring and alerting capabilities. This will be done by replacing the Raspberry Pi Pico board with another board that has embedded wireless communication, such as Pico W or WH. An external module will likely not be chosen due to the higher consumption the system will have, when compared to an processing unit with embedded transmission module, as shown on Assignment 1.

The system autonomy was out of the scope of this prototype, that had more focus on the threshold-based algorithm and the first hardware design. In all the tests, due to the need of collecting data for further analysis, a serial communication was implemented, which led to the system being powered by the USB port while transmitting the data.

As stated in limitations, this work only tested four scenarios, and from those, only two were non-fall scenarios. The next phase will involve testing the system under a larger range of conditions, including dynamic daily activities such as jogging, walking, jumping, and climbing stairs. Also, situations that change the magnitude acceleration even when the body is static, like traveling by bus or car will be considered, along with false fall scenarios where the user loses balance but recovers equilibrium. These tests will help refine the algorithm and reduce false positives in real-world environments.

The original idea of the fall detection algorithm was to have a threshold-based filter and a machine learning fall detector. Due to the difficulty on finding a good dataset to train the model, it was decided that the first prototype would be fully threshold-based. The second phase will be to compare performance and execution time of both algorithms to check the best approach. Recently, we found the MobiFall dataset [46], that gathers accelerometer and gyroscope data for classify in two classes: activity of daily life (ADL) and fall, and finally could train a model. The model is a hybrid model composed by a convolutional neural network and a bidirectional long short-term memory, CNN-BiLSTM archi-

tecture, a well-know architecture for time-series forecasting task [47]. The training was done in Python using Tensorflow, and the results showed an overall accuracy of 0.96 with recall of 0.92. The confusion matrix of this model can be seen on Figure 15.

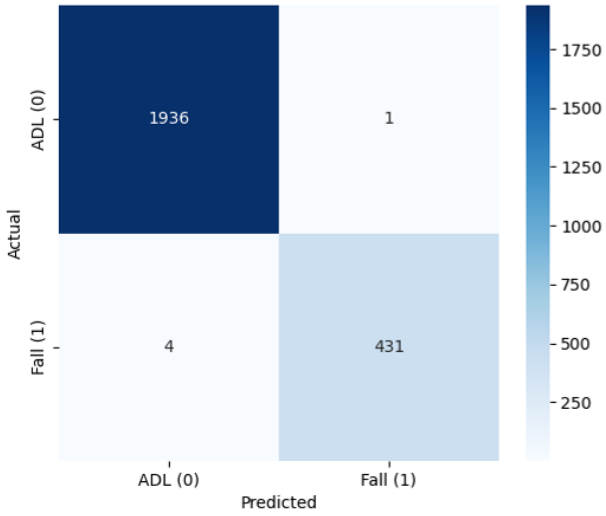


FIGURE 15. Confusion matrix of proposed CNN-BiLSTM model

Due to the shortage of time, it was not possible to implement this model on the Raspberry code, and will be another feature for the next step.

VIII. CONCLUSION

In conclusion, the fall detection prototype developed offers a promising solution for fall detecting, particularly addressed for vurnerable populations such as elderly or inidividuals working in hazardous environment. The integration of an IMU and a thresold-based algorithm enables the system to effectively identify falls. The deisgn of the module is lightweight, non-invasive, and stable, making it suitable for everyday activity. However, several limitations were ecountered during the testing phase affecting its performance. The external placement of the sensor introduced slight instability, leading to minor inaccuracies in the data. While the algorithm demonstrated reliable performance, its response time and tests on small scale scenarios, need improvements. Future works include improvements in sensor integration, algorithm refinement, and execution speed that are necessary for enhancing detection accuracy and reducing false positives. As well as the integration of the wireless communication and machine learning in the post-fall detection.

REFERENCES

[1] R. Hollinghurst, N. Williams, R. Pedrick-Case, L. North, S. Long, R. Fry, and J. Hollinghurst, "Annual risk of falls resulting in emergency department and hospital attendances for older people: an observational study of 781,081 individuals living in wales (united kingdom) including deprivation, frailty and dementia diagnoses between 2010 and 2020," *Age and ageing*, vol. 51, no. 8, p. afac176, 2022.

[2] P. G. Farup, K. Hestad, and K. Engedal, "Falls in persons with cognitive impairment—incidence and characteristics of the fallers," *Geriatrics*, vol. 9, no. 6, p. 168, 2024.

[3] A. J. Ferrari, D. F. Santomauro, A. Aali, Y. H. Abate, C. Abbafati, H. Abbastabar, S. Abd ElHafeez, M. Abdelmasseh, S. Abd-Elsalam, A. Abdollahi, *et al.*, "Global incidence, prevalence, years lived with disability (ylds), disability-adjusted life-years (dalys), and healthy life expectancy (hale) for 371 diseases and injuries in 204 countries and territories and 811 subnational locations, 1990–2021: a systematic analysis for the global burden of disease study 2021," *The Lancet*, vol. 403, no. 10440, pp. 2133–2161, 2024.

[4] J. A. Haagsma, B. F. Olij, M. Majdan, E. F. Van Beeck, T. Vos, C. D. Castle, Z. V. Dingels, J. T. Fox, E. B. Hamilton, Z. Liu, *et al.*, "Falls in older aged adults in 22 european countries: incidence, mortality and burden of disease from 1990 to 2017," *Injury prevention*, vol. 26, no. Suppl 2, pp. i67–i74, 2020.

[5] Statistics Norway, "Care services." https://www.ssb.no/en/helse/helsetjenester/statistikk/sjukeheimar-heimetenester-og-andre-omsorgstenester, 2025. Accessed: 2025-11-16.

[6] J.-E. Gjertsen, J. Fevang, T. Vinje, L. B. Engesæter, K. Steindal, and O. Furnes, "The norwegian hip fracture register/nasjonalt hoftebrud-dregister," *Norsk epidemiologi*, vol. 16, no. 2, 2006.

[7] T. K. Omsland, K. Holvik, H. E. Meyer, J. R. Center, N. Emaus, G. S. Tell, B. Schei, A. Tverdal, C. G. Gjesdal, G. Grimnes, *et al.*, "Hip fractures in norway 1999–2008: time trends in total incidence and second hip fracture rates. a norepos study," *European journal of epidemiology*, vol. 27, no. 10, pp. 807–814, 2012.

[8] D. Saplacan and J. Tørresen, "Robots as welfare technologies to reduce falls amongst older adults: an explorative study from norway," in *International Conference on Human-Computer Interaction*, pp. 88–106, Springer, 2022.

[9] World Health Organization, "Ageing and health." https://www.who.int/news-room/fact-sheets/detail/ageing-and-health, October 2025. Accessed: 2025-10-08.

[10] J. M. Kinge, H. Øien, J. L. Dieleman, B.-A. Reme, A. K. S. Knudsen, G. Godager, G. Selbæk, J. C. Frich, E. Barış, C. J. Murray, *et al.*, "Forecasting total and cause-specific health expenditures for 116 health conditions in norway, 2022–2050," *BMC medicine*, vol. 23, no. 1, p. 116, 2025.

[11] V. Andersen, V. R. Gurigard, J. A. Holter, and T. Wisborg, "Geographical risk of fatal and non-fatal injuries among adults in norway," *Injury*, vol. 52, no. 10, pp. 2855–2862, 2021.

[12] I. M. W. Nilsbakken, T. Wisborg, S. Sollid, and E. Jeppesen, "Emcc dispatch priority for trauma patients in norway: a retrospective cohort study," *Scandinavian Journal of Trauma, Resuscitation and Emergency Medicine*, vol. 33, no. 1, p. 83, 2025.

[13] S. Winge and E. Albrechtsen, "Accident types and barrier failures in the construction industry," *Safety science*, vol. 105, pp. 158–166, 2018.

[14] S. Holen and I. M. Holmen, "Occupational injuries and fatalities in norwegian fish farming," *Journal of Agromedicine*, vol. 30, no. 2, pp. 319–329, 2025.

[15] R. Igual, C. Medrano, and I. Plaza, "Challenges, issues and trends in fall detection systems," *Biomedical engineering online*, vol. 12, no. 1, p. 66, 2013.

[16] P. Vallabh and R. Malekian, "Fall detection monitoring systems: a comprehensive review," *Journal of Ambient Intelligence and Humanized Computing*, vol. 9, no. 6, pp. 1809–1833, 2018.

[17] A. Núñez-Marcos, G. Azkune, and I. Arganda-Carreras, "Vision-based fall detection with convolutional neural networks," *Wireless communications and mobile computing*, vol. 2017, no. 1, p. 9474806, 2017.

[18] T. Lee and A. Mihailidis, "An intelligent emergency response system: preliminary development and testing of automated fall detection," *Journal of telemedicine and telecare*, vol. 11, no. 4, pp. 194–198, 2005.

[19] M. Hua, Y. Nan, and S. Lian, "Falls prediction based on body keypoints and seq2seq architecture," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pp. 0–0, 2019.

[20] G. Mastorakis and D. Makris, "Fall detection system using kinect's infrared sensor," *Journal of Real-Time Image Processing*, vol. 9, no. 4, pp. 635–646, 2014.

[21] Y. Li, K. Ho, and M. Popescu, "A microphone array system for automatic fall detection," *IEEE Transactions on Biomedical Engineering*, vol. 59, no. 5, pp. 1291–1301, 2012.

[22] L. Tanutama, H. Wijaya, and D. Ardianti, "Elderly fall detection and warning system," in *IOP Conference Series: Earth and Environmental Science*, vol. 794, p. 012131, IOP Publishing, 2021.

[23] A. Fanca, A. Puscasiu, D.-I. Gota, and H. Valean, "Methods to minimize false detection in accidental fall warning systems," in *2019 23rd International Conference on System Theory, Control and Computing (ICSTCC)*, pp. 851–855, IEEE, 2019.

[24] A. T. Özdemir, "An analysis on sensor locations of the human body for wearable fall detection devices: Principles and practice," *Sensors*, vol. 16, no. 8, p. 1161, 2016.

[25] T. N. Gia, V. K. Sarker, I. Tcarenko, A. M. Rahmani, T. Westerlund, P. Liljeberg, and H. Tenhunen, "Energy efficient wearable sensor node for iot-based fall detection systems," *Microprocessors and Microsystems*, vol. 56, pp. 34–46, 2018.

[26] R. M. Gibson, A. Amira, N. Ramzan, P. Casaseca-de-la Higuera, and Z. Pervez, "Multiple comparator classifier framework for accelerometer-based fall detection and diagnostic," *Applied Soft Computing*, vol. 39, pp. 94–103, 2016.

[27] Q. Li, J. A. Stankovic, M. A. Hanson, A. T. Barth, J. Lach, and G. Zhou, "Accurate, fast fall detection using gyroscopes and accelerometer-derived posture information," in *2009 sixth international workshop on wearable and implantable body sensor networks*, pp. 138–143, IEEE, 2009.

[28] A. Sorvala, E. Alasaarela, H. Sorvoja, and R. Myllylä, "A two-threshold fall detection algorithm for reducing false alarms," in *2012 6th International Symposium on Medical Information and Communication Technology (ISMICT)*, pp. 1–4, IEEE, 2012.

[29] D. B. Apatsara Kitkamchon, Kan Dissorn, "A wearable fall detection device," *The 2021 Biomedical Engineering International Conference*, vol. 21, no. 6, 2021.

[30] S. Subramaniam, A. I. Faisal, and M. J. Deen, "Wearable sensor systems for fall risk assessment: A review," *Frontiers in digital health*, vol. 4, p. 921506, 2022.

[31] M. Villa and E. Casilari, "Energy-efficient fall-detection system using lora and hybrid algorithms," *Biomimetics*, vol. 10, no. 5, p. 313, 2025.

[32] A. H. M. Saod, A. A. Mustafa, Z. H. C. Soh, S. A. Ramlan, and N. A. Harron, "Dataset and system design for orthopedic walker fall detection and activity logging usingwith automated notification," in *2021 11th IEEE international conference on control system, computing and engineering (ICCSCE)*, pp. 182–187, IEEE, 2021.

[33] L. Liu, Y. Hou, J. He, J. Lungu, and R. Dong, "An energy-efficient fall detection method based on fd-dnn for elderly people," *Sensors*, vol. 20, no. 15, p. 4192, 2020.

[34] S. Li, "Fall detection with wrist-worn watch by observations in statistics of acceleration," *IEEE Access*, vol. 11, pp. 19567–19578, 2023.

[35] InvenSense, Inc. / TDK Sensor System Business, *MPU-6000 and MPU-6050 Product Specification (Revision 3.4)*. InvenSense, Inc., 1197 Borregas Ave., Sunnyvale, CA 94089, USA, Aug. 2013. Document No. PS-MPU-6000A-00, Release Date: 08/19/2013. Available at https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf.

[36] M. Pokydko, O. Oliinyk, and V. Tymchenko, "Mems gyroscope based on mpu-6050 sensor and atmega328 microcontroller," in *2024 IEEE 7th International Conference on Smart Technologies in Power Engineering and Electronics (STEE)*, pp. TT3–39, IEEE, 2024.

[37] W. Saadeh, S. A. Butt, and M. A. B. Altaf, "A patient-specific single sensor iot-based wearable fall prediction and detection system," *IEEE transactions on neural systems and rehabilitation engineering*, vol. 27, no. 5, pp. 995–1003, 2019.

[38] M. Thothadri, "An analysis on clock speeds in raspberry pi pico and arduino uno microcontrollers," *American Journal of Engineering and Technology Management*, vol. 6, no. 3, pp. 41–46, 2021.

[39] D. Loker, "Embedded systems using the raspberry pi pico," in *2022 ASEE Annual Conference & Exposition*, 2022.

[40] A. Knörig, R. Wettach, and J. Cohen, "Fritzing: a tool for advancing electronic prototyping for designers," in *Proceedings of the 3rd international conference on tangible and embedded interaction*, pp. 351–358, 2009.

[41] R. P. Foundation, *Getting Started with the Raspberry Pi Pico (RP-008276 DS-1)*. Raspberry Pi Foundation, 2021. Release date: 1 February 2021.

[42] R. P. Ltd., *Raspberry Pi Pico-series C/C++ SDK (RP-008354 DS-1)*, 2021. Initial release: 21 January 2021.

[43] F. Lundh, "An introduction to tkinter," *URL: www. pythonware. com/library/tkinter/introduction/index. htm*, vol. 539, p. 540, 1999.

[44] G. Welch, G. Bishop, *et al.*, "An introduction to the kalman filter," 1995.

[45] P. S. Kumar and U. Vignesh, "Analysis of the kalman filter with the mpu6050 accelerometer and gyroscope," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp. 1–6, IEEE, 2024.

[46] G. Vavoulas, M. Pediaditis, C. Chatzaki, E. G. Spanakis, and M. Tsiknakis, "The mobifall dataset: Fall detection and classification with a smartphone," *International Journal of Monitoring and Surveillance Technologies Research (IJMSTR)*, vol. 2, no. 1, pp. 44–56, 2014.

[47] M. Rhanoui, M. Mikram, S. Yousfi, and S. Barzali, "A cnn-bilstm model for document-level sentiment analysis," *Machine Learning and Knowledge Extraction*, vol. 1, no. 3, pp. 832–847, 2019.

MARCOS MOURAcontributed on the **prototype** with designing the circuit, designing, printing and modifying the wearable cover, programming the Raspberry Pi Pico and the monitoring interface. Regarding the **experiments**, he contributed on performing the tests and generating the graphs, using MATLAB. On **this report** he contributed on Sections I, III.A, III.B, III.C, III.D, III.E, IV.A, V and VII.B. He also contributed on Tables 1 and 2, and on Figures from 1 to15.

MARCO CHIMIENTI contributed in the **algorithm**, with the thresholding method, the conceptualization of the **design**. Also in the **experiments**, testing the fall scenarios. On the report he contributed on IV, IV.B, VI, VII, VII.A, VIII.

AGATA BARBARA BRECZKO contributed in decisions regarding the search for innovations introduced by us. Also in selection of sensors and microcontrollers for the **prototype**. In **this report**, she contributed on Sections II.